

SESSION 2 · PRE-READ

Mathematical Foundations for AI Systems

WORKING TIME

About 90 minutes, read at a steady pace with a pen

COVERS

Four parts, fourteen topics — the whole session, in outline

PREPARED BY

Mr. Shridhar Galande

Every idea in Saturday's session gets an intuition, a worked number and a business scenario. This pre-read gives you the first pass at all fourteen topics, so class time can be spent on discussion rather than first exposure.

There is nothing to submit. Read it once, tick the checklists honestly, and bring the boxes you could not tick.

HOW TO USE THIS

Read for recognition, not mastery

You do not need to be able to reproduce any of this from memory on Saturday. You need the symbols to look familiar, so that when a formula goes up on screen your attention is on what it means rather than on decoding it.

Each topic below follows the same shape: a plain-language intuition, the formula with every term named, a worked example with real numbers, and where the idea shows up in a business system.

All the arithmetic is done for you. If you can follow a step, that is enough — you are not expected to have produced it.

ASSUMED BEFORE YOU START

- Fractions, percentages and decimals
- Powers and square roots
- Rearranging a simple equation
- Reading a table of numbers

Nothing beyond school algebra is assumed.

Calculus, matrices and probability notation are all introduced from scratch.

THE FOUR PARTS

1. Counting & Probability — what could happen, and how likely
2. Vectors & Meaning — turning language into geometry
3. Matrices & Tensors — the arithmetic inside a transformer
4. Learning & Decisions — how models improve and choose

WHAT THE MARKS MEAN

Formula card

The expression, then every term named on its own line. Red rule at the top.

Worked

Real numbers, every step shown, answer included.

In business

One place the idea earns its keep in a working system.

PART ONE

Counting & Probability

Before a system can choose well, it has to know what could happen and how likely each outcome is. Topics 1 to 3.

TOPIC 01

Counting & combinations

You cannot optimise a choice until you know how many choices exist. Two questions separate the two tools: does order matter? Ranking three finalists first, second and third is a **permutation**. Picking three people for a team, where nobody is first, is a **combination**.

$$C(n, r) = n! / (r! \cdot (n - r)!)$$

The number of ways to choose r things from n , when order does not matter

$n!$	Every arrangement of all n items: $n \times (n-1) \times \dots \times 1$
$r!$	Divides out the orderings <i>inside</i> the group you picked — a team is a team however you list it
$(n - r)!$	Divides out the orderings among the items you left behind, which you never cared about

WORKED · FIVE TOPPINGS, CHOOSE THREE

$$C(5, 3) = 5! / (3! \cdot 2!) = 120 / (6 \times 2) = \mathbf{10}$$

Ten sundaes, and you can write them all out. With toppings A B C D E: ABC, ABD, ABE, ACD, ACE, ADE, BCD, BCE, BDE, CDE. Note that $5! / 2! = 60$ would be the answer if order mattered — six podium orders for each of the ten teams.

IN BUSINESS A marketing team with eight channels wants to test every three-channel mix: $C(8, 3) = 56$ experiments. The same multiplication is why a hyperparameter sweep explodes — four learning rates $\times$ three batch sizes $\times$ three depths is 36 training runs, not ten.

TOPIC 02

Probability foundations

A probability is a fraction of a whole, which is why it always sits between 0 and 1. A **conditional** probability is the same fraction with a smaller denominator — you have been told something, so fewer outcomes remain in play. **Bayes' theorem** reverses the direction of a conditional: it turns "how often does this test fire when the patient is ill" into "how likely is illness given that the test fired".

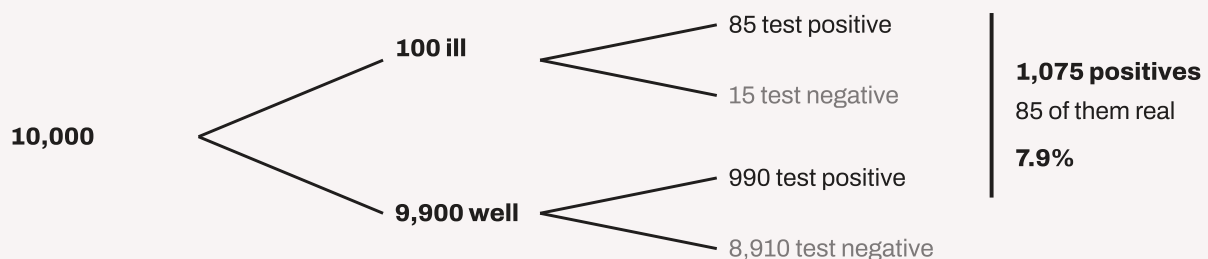
$$P(A | B) = P(B | A) \cdot P(A) / P(B)$$

Read the bar as "given". Bayes' theorem

P(A)	The <i>prior</i> — what you believed before the evidence arrived. Often the base rate
P(B A)	The <i>likelihood</i> — how often this evidence appears when A really is true
P(B)	The <i>evidence</i> — how often that evidence appears at all, true positives and false ones together
P(A B)	The <i>posterior</i> — the updated belief. This is what you actually wanted

WORKED · AN 85% ACCURATE TEST, ON A 1% DISEASE

Ratios are hard to hold in the head, so count people instead. Take 10,000 of them.



$P(\text{ill} | \text{positive}) = 85 / 1,075 = \mathbf{0.079}$. A positive result on an 85% accurate test still leaves you more than 90% likely to be well, because the healthy group is so much larger that its small error rate produces most of the positives.

IN BUSINESS A spam filter runs exactly this calculation on every message, and it is an easier problem precisely because the base rate is high — roughly half of all mail. Fraud detection is the hard case: the base rate is a fraction of a percent, so false positives swamp real catches unless the model is far more precise than it sounds.

TOPIC 03

Distributions & sampling

Two questions cover most of what a business asks. *Out of a fixed number of tries, how many will succeed?* — that is the **binomial**. *How many events will arrive in an interval, when there was never a fixed number of tries?* — that is the **Poisson**. The test is simply whether you can name n .

$$P(X = k) = C(n, k) \cdot p^k \cdot (1 - p)^{n - k}$$

Binomial — counting $\times$ probability

$C(n, k)$	How many different orders the k successes could arrive in — the combination from Topic 01
p^k	The chance of those k successes, multiplied together
$(1-p)^{n-k}$	The chance the remaining tries all fail

$$P(X = k) = \lambda^k \cdot e^{-\lambda} / k!$$

Poisson — no n anywhere in it

λ	The average number of events per interval — the only input you need
k	The number of events you are asking about

WORKED · EIGHT SURVEYS, EXACTLY THREE YESES

A transit authority surveys eight riders; historically 60% say yes. $P(X = 3) = C(8, 3) \times 0.6^3 \times 0.4^5 = 56 \times 0.216 \times 0.01024 = \mathbf{0.124}$, about a one-in-eight chance.

Poisson. A station averages two broken ticket machines ($\lambda = 2$). $P(X = 3) = 2^3 \times e^{-2} / 3! = 8 \times 0.1353 / 6 = \mathbf{0.180}$. Nobody counted how many machines could have failed — only the average rate was known.

Sampling. Error shrinks as $1/\sqrt{n}$. Going from 100 to 400 samples halves your margin of error; halving it again needs 1,600. Four times the data buys twice the confidence, and no more.

IN BUSINESS Call-centre and support staffing is the classic Poisson problem: you know the average tickets per hour, and you need the probability of a spike. A/B test sizing is the sampling problem — the square-root law is why a test that needs a small lift detected needs a very large audience.

PART ONE · CAN YOU DO THIS?

- ☐ Say whether a problem needs a permutation or a combination
- ☐ Read $P(A | B)$ aloud and say what it means
- ☐ Choose between binomial and Poisson for a given question
- ☐ Explain why $C(n,r)$ divides by both $r!$ and $(n-r)!$
- ☐ Explain why a rare condition makes an accurate test unreliable
- ☐ State how much data it takes to halve a margin of error

PART TWO

Vectors & Meaning

Turning words, products and people into positions in space, so that "related" becomes something you can measure. Topics 4 to 6.

TOPIC 04

Vectors, embeddings & semantic space

A **vector** is just an ordered list of numbers. An **embedding** is a vector chosen so that its position carries meaning — the way [latitude, longitude] pins a place on Earth, an embedding's few hundred numbers pin a concept in meaning-space. Nearby coordinates mean related meaning, and that is the entire premise behind search, recommendations and retrieval.

$$d(\mathbf{A}, \mathbf{B}) = \sqrt{(\sum_i (\mathbf{x}_i - \mathbf{y}_i)^2)}$$

Euclidean distance — Pythagoras, with as many terms as you have dimensions

$\mathbf{x}_i - \mathbf{y}_i$	The gap between the two vectors on one dimension
$(\dots)^2$	Squared, so that a gap counts the same whichever direction it runs
$\sum_i$	Add those squared gaps across every dimension. Sigma just means "add up"
$\sqrt{}$	The square root brings the answer back to the original scale

WORKED · TWO CUSTOMER PROFILES

$\mathbf{A} = [2, 3]$ and $\mathbf{B} = [5, 7]$. Gaps are 3 and 4. Squared: 9 and 16. Sum: 25. $d(\mathbf{A}, \mathbf{B}) = \sqrt{25} = 5$.

Real embeddings do this over 384, 768 or 1,536 dimensions instead of two. The arithmetic does not change — there are simply more terms in the sum. Extra dimensions are what let the model keep "bank" the riverside and "bank" the institution in different places.

IN BUSINESS "Customers also bought" is embeddings and nothing else: each product's description, category and reviews become a vector, and the nearest vectors become the recommendations.

TOPIC 05

Similarity metrics & vector search

Distance answers "how far apart". **Cosine similarity** answers "which way are they pointing" — two hikers can cover very different distances in exactly the same direction. For text that distinction matters, because a long document and a short one about the same subject should score as similar even though their vectors have very different lengths.

$$\cos(\mathbf{A}, \mathbf{B}) = (\mathbf{A} \cdot \mathbf{B}) / (\|\mathbf{A}\| \cdot \|\mathbf{B}\|)$$

Runs from -1 (opposite) through 0 (unrelated) to 1 (same direction)

$\mathbf{A} \cdot \mathbf{B}$	The dot product: multiply the pairs, add the results. Large when both vectors are large in the same places
$\ \mathbf{A}\ $	The length (magnitude) of A: $\sqrt{a_1^2 + a_2^2 + \dots}$
$\div \ \mathbf{A}\ \ \mathbf{B}\ $	Dividing by both lengths strips out magnitude, leaving only direction

WORKED · A CLINICAL SEARCH QUERY

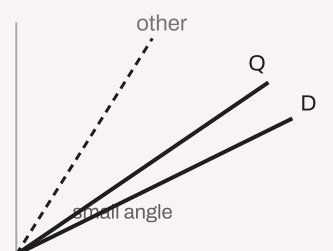
$\mathbf{Q} = [0.85, 0.80, 0.10]$ — "heart attack treatment". $\mathbf{D} = [0.90, 0.80, 0.10]$ — "cardiac arrest emergency care".

Dot product: $0.765 + 0.640 + 0.010 = 1.415$

$\|\mathbf{Q}\| = \sqrt{1.3725} = 1.1716$ $\|\mathbf{D}\| = \sqrt{1.4600} = 1.2083$

$\cos = 1.415 / 1.4157 = \mathbf{0.9995}$

Near-identical direction, so the retriever ranks D first. A document scoring 0.31 on the same query is not a weaker match — it is a different subject.



IN BUSINESS Every "similar items" rail and every RAG retrieval step is a cosine ranking over a vector index. Rule of thumb: text and embeddings want cosine; physical measurements in real units — price, weight, distance — want Euclidean.

TOPIC 06

Graphs & relationships

Embeddings represent meaning by position. Graphs represent it by explicit relationship. Five words give you the whole vocabulary, and a graph turns into arithmetic the moment you write it as a matrix.

Node	A thing — a customer, a document, a step in a plan
Edge	A relationship between two nodes
Direction	Whether the relationship runs one way or both
Weight	How strong or costly that edge is
Path	A route of edges from one node to another

$A_{ij} = 1$ if an edge joins i and j , else 0

The **adjacency matrix**. Once a graph is a grid of numbers, every matrix operation from Part 3 applies to it — which is how graph neural networks work at all.

IN BUSINESS Fraud rings show up as unusually dense clusters of edges between accounts. Knowledge graphs give a retrieval system facts with named relationships rather than loose text. Agent routing is a graph too — a plan is a path through a set of possible steps.

PART TWO · CAN YOU DO THIS?

- | | |
|--|---|
| ☐ Explain what an embedding is to a non-technical colleague | ☐ Compute the distance between two two-dimensional vectors |
| ☐ Take a dot product without looking up the definition | ☐ Say why cosine, not distance, is used for text search |
| ☐ Say what a cosine score of 0.99, 0.5 and -0.2 each mean | ☐ Name the five parts of a graph |

PART THREE

Matrices & Tensors

The arithmetic a transformer actually performs, and the two functions that keep its numbers manageable. Topics 7 and 8.

TOPIC 07

Matrices, tensors & transformer operations

A scalar is one number, a vector is a row of them, a matrix is a grid, and a tensor is a stack of grids. Each step adds one axis; nothing new is introduced. A matrix's **shape** is "rows × columns", and almost every practical error in this field is a shape mismatch.

The shape rule. $(m \times n) \times (n \times p) \rightarrow (m \times p)$. The inner numbers must match, and the outer numbers give you the result. If the inner numbers differ, the multiplication does not exist.

$$(A \times B)_{i,j} = \sum_k A_{i,k} \cdot B_{k,j}$$

Matrix multiplication — one dot product per output cell

i, j	Which cell of the answer you are filling in — row i, column j
$A_{i,k}$	Row i of the left matrix, walked across
$B_{k,j}$	Column j of the right matrix, walked down
$\sum_k$	Multiply those pairs and add — the dot product again, once per cell

WORKED · A 2×2 MULTIPLICATION, ALL FOUR CELLS

$$\begin{array}{|c|c|} \hline 1 & 2 \\ \hline 3 & 4 \\ \hline \end{array} \times \begin{array}{|c|c|} \hline 5 & 6 \\ \hline 7 & 8 \\ \hline \end{array} = \begin{array}{|c|c|} \hline \mathbf{19} & \mathbf{22} \\ \hline \mathbf{43} & \mathbf{50} \\ \hline \end{array}$$

Row 1 $\times$ Col 1: $(1 \times 5) + (2 \times 7) = 19$ Row 1 $\times$ Col 2: $(1 \times 6) + (2 \times 8) = 22$ Row 2 $\times$ Col 1: $(3 \times 5) + (4 \times 7) = 43$ Row 2 $\times$ Col 2: $(3 \times 6) + (4 \times 8) = 50$

The operations worth recognising. Addition and subtraction work element by element and need identical shapes. Scalar multiplication scales every entry — exactly what a learning rate does to a gradient. The **transpose** A^T flips rows into columns. The **identity** I is matrix multiplication's number 1: $A \times I = A$. The **inverse** A^{-1} undoes A . The **determinant** is one number saying how much a matrix stretches or squashes space; zero means it collapsed a dimension and cannot be undone.

$Q \times K^T$ — what attention computes. Take three tokens with two dimensions each, $Q = K = [[1,0], [0,1],[1,1]]$ — call the rows "cat", "sat", "mat". Multiplying Q by K^T scores every token against every other, giving a 3×3 grid: $[[1,0,1],[0,1,1],[1,1,2]]$. The diagonal is each token against itself; "mat" scores 2 with itself because it is active on both dimensions. That grid, softmaxed, is an attention pattern. The transpose is what makes the shapes line up.

IN BUSINESS A loan model scores applicants on income, credit and history weighted 0.5, 0.3 and 0.2 — a matrix of applicants $\times$ features times a weight vector, scoring the whole book in one operation. Batching works the same way, and is why serving a thousand requests together costs far less per request than serving them one at a time.

TOPIC 08

Logs & exponentials

A logarithm asks: how many times do I multiply the base by itself to get here? $\log_{10}(1000) = 3$, because $10 \times 10 \times 10 = 1000$. It appears everywhere in AI for one practical reason — it turns multiplication into addition, which keeps very small probabilities from vanishing. The exponential e^x runs the other way and has the property softmax depends on: it is positive for every input, however negative.

$\log(a \cdot b) = \log(a) + \log(b)$

The property every log-likelihood in machine learning rests on

log	Base 10 unless stated. ln is base $e \approx 2.718$, the default in machine learning; $\log_2$ gives answers in bits
$a \cdot b$	A chain of probabilities multiplied together — each one below 1, so the product collapses fast
+	Addition instead, which a computer can hold at any length without losing the number

WORKED · WHY A MODEL SCORES IN LOGS

Three token probabilities: $0.001 \times 0.002 \times 0.005 = 0.00000001$. Over a thousand tokens, that product underflows to zero and the model can no longer tell a good sequence from a bad one.

In logs: $(-3) + (-2.70) + (-2.30) = \mathbf{-8.00}$. Same information, no collapse, and it stays legible however long the sequence runs.

IN BUSINESS Log-likelihood is how any scoring model ranks candidates. Log scales are also why growth and cost curves get plotted logarithmically — a straight line on a log axis is a constant multiplication, not a constant increase.

PART THREE · CAN YOU DO THIS?

- | | |
|--|--|
| ☐ Say the shape of a matrix and whether two can be multiplied | ☐ Multiply two 2×2 matrices by hand |
| ☐ Explain what a transpose does and why attention needs one | ☐ Say what the identity and the inverse are for |
| ☐ Read a tensor shape like (32, 128, 768) and name each axis | ☐ Explain why probabilities are added as logs, not multiplied |

PART FOUR

Learning & Decisions

How a model improves, how it turns scores into choices, and how you tell whether any of it worked. Topics 9 to 14.

TOPIC 09

Calculus, gradients & learning

You are walking down a hill in fog. You cannot see the bottom, but you can feel which way the ground slopes under your feet, so you take a small step that way and feel again. That is gradient descent, with error in place of height. A **derivative** is just rise over run — the slope between two points on a curve, as those points slide together.

$$w_{\text{new}} = w_{\text{old}} - \alpha \cdot \partial L / \partial w$$

The update rule. Every model that has ever been trained ran this, millions of times

w	A weight — one of the numbers the model is allowed to change
L	The loss: a single number measuring how wrong the model currently is
$\partial L / \partial w$	The gradient — how much the loss changes if this one weight nudges up. Points uphill
–	The minus sign is the whole trick: the gradient points uphill, so subtract it to go down
α	The learning rate — how big a step to take. Too small and it crawls; too large and it overshoots

WORKED · A MODEL WITH EXACTLY ONE WEIGHT

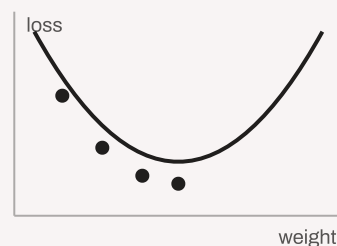
Predict $y = w \cdot x$, with $w = 2$ and $x = 3$. The true answer is 8.

Prediction: $2 \times 3 = 6$ Loss: $(6 - 8)^2 = 4$

Gradient: $2 \times (6 - 8) \times 3 = -12$

With $\alpha = 0.01$: $w = 2 - 0.01 \times (-12) = \mathbf{2.12}$

New prediction 6.36, new loss 2.69. One step, a third of the error gone. Repeat a few thousand times and w converges near 2.67.



IN BUSINESS A house-price model predicting \$130K for a house that sold for \$100K has a \$30K error, and this loop is what shrinks it. Fine-tuning and RLHF are the same loop with a different loss function — the mechanism does not change, only what counts as wrong.

TOPIC 10

Probability & token prediction

Given "The capital of France is ___", a language model does not retrieve a fact. It produces a raw score for every token in its vocabulary and converts those scores into a probability distribution.

Softmax is the conversion — a panel of judges' raw marks turned into percentages of the vote.

$$\text{softmax}(z_i) = e^{z_i} / \sum_j e^{z_j}$$

Any list of numbers into a set of probabilities that sum to 1

z_i	The raw score, or <i>logit</i> , for one candidate. Can be negative
e^{z_i}	Exponentiating forces it positive and exaggerates the gaps between scores
$\sum_j e^{z_j}$	The same for every candidate, added up — the denominator that makes the results sum to 1
τ	<i>Temperature</i> , when present: each logit is divided by it first. Below 1 sharpens, above 1 flattens

WORKED · FOUR CANDIDATE TOKENS

TOKEN	LOGIT Z	e^Z	PROBABILITY
Paris	3.2	24.53	0.825
London	1.1	3.00	0.101
Berlin	0.6	1.82	0.061
Apple	-1.0	0.37	0.012

Denominator 29.72. A logit gap of 2.1 became a probability ratio of roughly eight to one — exponentiating is what turns a modest lead into a confident answer.

IN BUSINESS Asked about a patient with fever and cough, a clinical assistant returns a distribution, not a diagnosis — 0.58 for one condition and 0.24 for another is a different situation from 0.94 and 0.02, and the interface should say so. Temperature is the dial: near 0 for extraction and classification, higher for drafting.

TOPIC 11

Information theory

Entropy is average surprise. A distribution with one dominant option has low entropy; one that finds everything equally plausible has high entropy. **Cross-entropy** measures the surprise of the true answer under the model's distribution — it is the loss function that trains every language model.

Perplexity is the same number made readable: roughly, how many options the model was effectively choosing between.

$$H = - \sum_i p_i \cdot \log(p_i)$$

Entropy. In $\log_2$ the answer is in bits

p_i	The probability of one outcome
$\log(p_i)$	Its surprise. Rare outcomes have large negative logs — they are surprising
$-\Sigma$	Weight each surprise by how often it happens, add, and flip the sign to make it positive

WORKED · A CONFIDENT MODEL VERSUS AN UNSURE ONE

Distribution [0.9, 0.1]: $H = -(0.9 \times -0.152 + 0.1 \times -3.322) = 0.137 + 0.332 = \mathbf{0.469 \text{ bits}}$.

Distribution [0.5, 0.5]: $H = -(0.5 \times -1 + 0.5 \times -1) = \mathbf{1.000 \text{ bit}}$ — the maximum for two options, and a model that has learned nothing useful.

Perplexity = 2^H (or e^H in natural logs). A perplexity of 12 means the model was, on average, torn between about twelve plausible next tokens.

IN BUSINESS Entropy is a live monitoring signal. When a deployed model's output entropy climbs, it is telling you it is meeting inputs it has no confident answer for — usually the first sign of drift, and a good trigger for routing to a human.

TOPIC 12

Numerical stability

e^{800} is not a large number as far as a 32-bit float is concerned — it is infinity. So every real softmax implementation subtracts the largest logit before exponentiating. The result is mathematically identical, because the same constant cancels top and bottom, but it never overflows.

$$\text{softmax}(z_i) = e^{(z_i - \max z)} / \sum_j e^{(z_j - \max z)}$$

The stable form — the one actually shipped

max z	The largest logit in the set. Subtracting it makes the biggest exponent exactly 0
$e^0 = 1$	So no term can exceed 1, and nothing overflows. Example: $z = [800, 799] \rightarrow [0, -1] \rightarrow 0.731$ and 0.269

FORMAT BITS WHERE IT IS USED

fp32	32	Full precision. Safe, slow, memory-hungry
fp16 / bf16	16	Standard for training and serving. Half the memory
int8	8	Quantised inference. A quarter of the memory, small accuracy cost

IN BUSINESS This table is behind every quantisation decision and every GPU memory estimate. A 7-billion-parameter model needs roughly 28GB in fp32, 14GB in fp16 and 7GB in int8 — which is the difference between a data-centre card and a workstation.

TOPIC 13

Evaluation

A classifier can be right in two ways and wrong in two ways. Naming those four cases is most of the work; every evaluation metric is then a ratio of two of them.

$$\text{precision} = \text{TP} / (\text{TP} + \text{FP}) \cdot \text{recall} = \text{TP} / (\text{TP} + \text{FN})$$

$F_1 = 2PR / (P + R)$ — the harmonic mean, when you need one number

TP	True positive — flagged, and it really was
FP	False positive — flagged, but it was not. A false alarm
FN	False negative — missed. The case that got through
Precision	When it flags something, how often is it right?
Recall	Of the real cases out there, how many did it catch?

WORKED · A FRAUD MODEL, SCORED HONESTLY

Out of 10,000 transactions, 100 are fraudulent. The model flags 180 and catches 80 of the real ones. So $TP = 80$, $FP = 100$, $FN = 20$, $TN = 9,800$.

Precision = $80/180 = \mathbf{0.44}$ Recall = $80/100 = \mathbf{0.80}$ $F_1 = \mathbf{0.57}$

Accuracy = $9,880/10,000 = \mathbf{98.8\%}$, which sounds excellent and means nothing — a model that flagged nothing at all would score 99%. On any rare event, accuracy is the metric to distrust.

IN BUSINESS Which error costs more decides the threshold. A missed fraud costs the transaction; a false alarm costs a blocked customer. In cancer screening the asymmetry is far sharper, and recall wins.

TOPIC 14

Optimization & agent decisioning

An agent is a scoring loop. Faced with several possible actions, it assigns each one a number — a **utility** — and takes the highest. Choosing a restaurant by weighing price, distance and rating into one score is the same operation.

$$\mathbf{a^* = \operatorname{argmax}_a U(a)} \quad \text{Return} = \sum_t r_t$$

Pick the best action now; judge the whole run by what it accumulated

U(a)	The utility of action a — one number combining everything you care about, weighted
argmax	"The argument that maximises" — returns <i>which</i> action scored highest, not the score itself
r_t	The reward received at step t. Can be negative — a penalty is just a negative reward
Σ_t	Added over the whole episode. This is what the agent is trained to maximise

WORKED · AN AGENT CHOOSING A TOOL

Asked to "generate quarterly financial analysis", the agent scores each available tool: database query 0.82, web search 0.44, calculator 0.31, image generator 0.05. **argmax** returns **database query** — the action, not the 0.82.

Its rewards for the episode might be +1.0 for a good retrieval, +0.5 for a correct summary, −2.0 for a fabricated figure and −0.1 per step. Return = −0.6, so despite two good actions the run scores negatively. The step penalty is what stops an agent looping forever.

IN BUSINESS Reward design shapes behaviour more than any prompt does. Dynamic pricing, ad auction bidding and route planning are all the same **argmax** over a utility you chose — which means whatever you forgot to price into U is the thing the system will happily destroy.

PART FOUR · CAN YOU DO THIS?

- | | |
|--|---|
| ☐ Explain gradient descent without using the word gradient | ☐ Say why the update rule subtracts rather than adds |
| ☐ Describe what softmax does to a list of raw scores | ☐ Choose a temperature for a task and defend it |
| ☐ Say what high output entropy tells you about a live model | ☐ Compute precision and recall from a confusion matrix |
| ☐ Explain why accuracy misleads on rare events | ☐ Read argmax aloud and say what it returns |

REFERENCE

Notation

Every symbol used in the session. Greek letters are the single biggest source of false difficulty — say the name out loud once and most of it goes away.

Σ	Sigma — add up everything that follows	I	Identity matrix — matrix multiplication's 1
Π	Pi — multiply everything that follows	$\det(A)$	Determinant — how much A stretches space
$n!$	Factorial — $n \times (n-1) \times \dots \times 1$	$\partial L / \partial w$	Partial derivative — the gradient of L in w
$C(n, r)$	Choose r from n, order ignored	∇	Nabla — all the partial derivatives at once
$P(A)$	The probability of A	α	Alpha — the learning rate
$P(A B)$	The probability of A given B	θ	Theta — the model's parameters, collectively
$E[X]$	Expected value — the long-run average of X	τ	Tau — temperature
μ	Mu — a mean	$e, \ln$	Euler's number ≈ 2.718 , and log to base e
σ, σ^2	Sigma — standard deviation, and variance	$\log_2$	Log base 2 — answers come out in bits
λ	Lambda — an average rate per interval	$\argmax$	Which option scores highest, not the score
$A \cdot B$	Dot product of two vectors	$\approx$	Approximately equal to
$\ v\ $	The length, or magnitude, of vector v	$\in$	Is a member of
$\mathbb{R}^n$	The space of n-dimensional vectors	$\hat{y}$	Y-hat — the predicted value, against true y
A^T	Transpose — rows become columns		
A^{-1}	Inverse — the matrix that undoes A		

REFERENCE

Glossary

Base rate. How common something is before any evidence. The most commonly ignored number in probability.

Batch. A group of inputs processed in one matrix operation rather than one at a time.

Binomial. The distribution for a fixed number of tries with a known success rate.

Combination. A selection where order does not matter.

Cosine similarity. A measure of how closely two vectors point in the same direction, ignoring their lengths.

Cross-entropy. The loss function used to train language models; the surprise of the true answer under the model's distribution.

Determinant. One number describing how much a matrix stretches or squashes the space it acts on.

Dot product. Multiply two vectors pairwise and add the results.

Embedding. A vector whose position in space carries meaning.

Entropy. Average surprise in a distribution. Low means confident.

Epoch. One full pass of the training data through the model.

Gradient. How much the loss changes when a weight changes. Points uphill, so training subtracts it.

Gradient descent. Repeatedly stepping weights in the direction that reduces loss.

Identity matrix. Ones on the diagonal, zeros elsewhere. Multiplying by it changes nothing.

Learning rate. The step size in gradient descent. Too small crawls, too large overshoots.

Logit. A raw, unnormalised score before softmax turns it into a probability.

Loss. A single number measuring how wrong the model currently is.

Perplexity. Entropy made readable — roughly how many options the model was choosing between.

Permutation. An arrangement where order matters.

Poisson. The distribution for events arriving at a known average rate, with no fixed number of tries.

Posterior / prior. Belief after the evidence, and before it.

Precision / recall. Of what it flagged, how much was right; of what was real, how much it caught.

Quantisation. Storing a model's numbers in fewer bits to cut memory and cost.

RAG. Retrieval-augmented generation — finding relevant documents by vector similarity, then generating an answer from them.

Shape. A matrix or tensor's dimensions, written rows × columns.

Softmax. Converts a list of raw scores into probabilities that sum to 1.

Temperature. A divisor applied to logits before softmax. Below 1 sharpens the distribution, above 1 flattens it.

Tensor. A stack of matrices — an array with three or more axes.

Transpose. Flipping a matrix so its rows become columns.

Utility. One number combining everything an agent cares about, used to rank possible actions.

Vector. An ordered list of numbers.

Weight. One of the adjustable numbers inside a model.

Bring on Saturday: this pre-read with the checklists marked, a pen, and the boxes you could not tick. The session runs through all fourteen topics again with worked exercises — this was the first pass, not the only one.